Classification dengan Ensemble Method & Class Imbalance

Deteksi Transaksi Penipuan Kartu Kredit pada Dataset Credit Card Fraud

- **Nama:** Rasya Arya Ramadhan
- **NRP:** 5054251019
- **Mata Kuliah:** Data Mining
- **Praktikum:** Tugas Praktikum 3

1. Pendahuluan

Praktikum ini nyoba ngebangun model klasifikasi buat deteksi transaksi kartu kredit yang penipuan. Yang bikin kasus ini menarik bukan cuma modelnya, tapi karakteristik datanya yang timpang parah. Transaksi fraud itu cuma 492 baris dari total 284.807 transaksi, atau sekitar 0,17 persen. Rasio kelasnya kira-kira 1 fraud tiap 578 transaksi normal.

Ketimpangan itu bikin pendekatan klasifikasi biasa nggak cocok. Model yang cuma dipaksa seakurat mungkin justru bakal condong ke kelas normal, karena dengan nebak semua transaksi sebagai normal aja dia udah bisa dapet akurasi 99,83 persen. Angkanya kelihatan bagus, tapi nggak ada satu pun penipuan yang ke-deteksi. Fenomena ini dikenal sebagai *accuracy paradox*.

Makanya sasaran praktikum ini ada dua: ngebangun model ensemble yang bagus, sekaligus ngasin penanganan yang tepat buat kelas minoritasnya. Dua algoritma ensemble yang dipakai adalah Random Forest sebagai perwakilan bagging dan XGBoost sebagai perwakilan boosting, masing-masing dijalankan dalam tiga skenario penanganan imbalance.

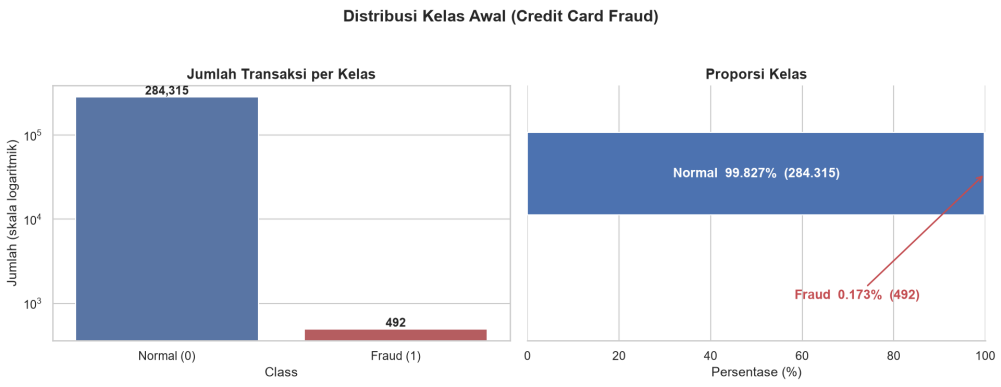
2. Dataset dan Distribusi Kelas

Datasetnya diambil dari Kaggle, isinya 284.807 baris dan 31 kolom. Kolom **V1** sampai **V28** itu hasil transformasi PCA, jadi namanya memang udah nggak bisa ditelusuri lagi ke arti aslinya karena data aslinya udah dianonimkan. Kolom yang masih bisa dibaca cuma tiga: **Time** (jarak detik dari transaksi pertama), **Amount** (nilai transaksi), dan **Class** (0 = normal, 1 = fraud).

Pemeriksaan awal nunjukkin nggak ada missing value sama sekali. Yang ada justru 1.081 baris duplikat, tapi tetep saya biarin aja karena nggak ngaruh ke tujuan praktikum ini dan malah sering muncul di dataset fraud yang asli.

Kelas	Jumlah	Persentase	Rasio terhadap mayoritas
0 (Normal)	284.315	99,8273 %	1 : 1
1 (Fraud)	492	0,1727 %	1 : 578

Tabel 1. Distribusi kelas pada dataset Credit Card Fraud.



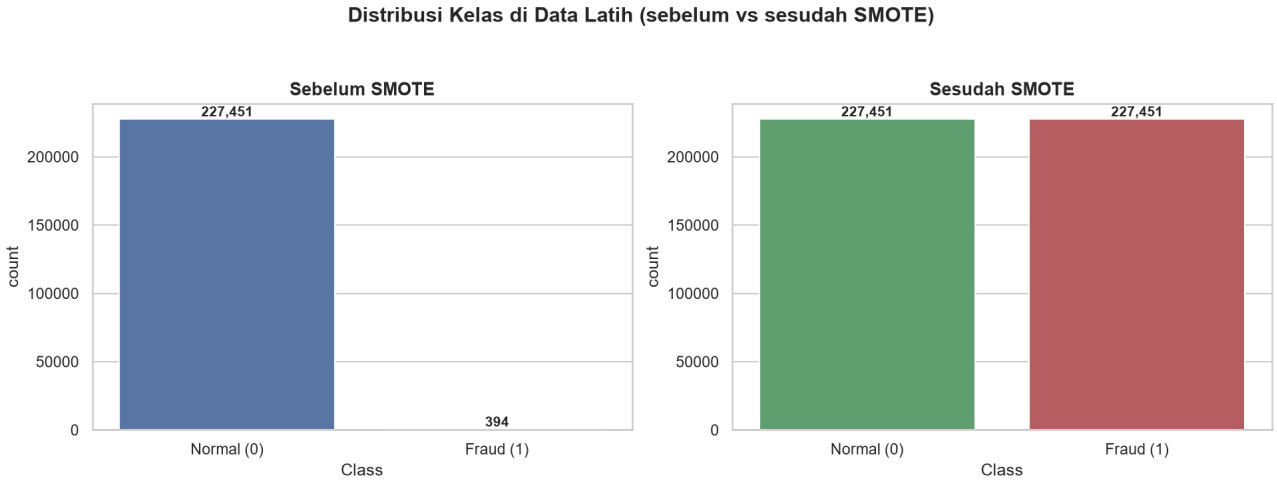
Gambar 1. Distribusi kelas awal. Grafik kiri pakai skala logaritmik biar kelas fraud yang cuma 492 baris tetep kelihatan.

3. Praproses dan Penanganan Class Imbalance

Dari kolom yang masih asli, yang saya scale cuma **Time** dan **Amount**. **Class** nggak disentuh karena itu labelnya. Kolom **V1** sampai **V28** juga nggak saya ubah karena hasil PCA biasanya udah punya rentang yang sebanding. Scaler yang dipakai **RobustScaler**, bukan **StandardScaler**, karena kolom Amount itu outliernya banyak banget, mayoritas transaksi nilainya kecil tapi ada yang beribu kali lipat. RobustScaler ngitung berdasarkan median dan IQR, jadi nggak gampang ketarik outlier kayak gitu.

Pembagian data dilakukan 80:20 dengan parameter **stratify=y**. Parameter ini penting banget di kasus ini, karena kelas fraudnya cuma 492 baris. Tanpa stratify, proporsi fraud di data latih dan data uji bisa nggak sama dan hasil evaluasinya langsung nggak stabil. Hasilnya data latih berisi 227.845 baris (394 fraud) dan data uji 56.962 baris (98 fraud).

Setelah splitting, baru diterapkan **SMOTE** (*Synthetic Minority Over-sampling Technique*) buat bikin data fraud sintetis sampai jumlahnya imbang, jadi 227.451 untuk masing-masing kelas. Resamplingnya **cuma dilakukan di data latih**. Kalau data uji juga di-SMOTE, hasil evaluasinya jadi nggak nyata, soalnya di dunia nyata kita nggak pernah punya data fraud yang jumlahnya udah dibikin imbang.



Gambar 2. Distribusi kelas di data latih sebelum dan sesudah SMOTE.

Selain SMOTE, dipakai juga pendekatan **class weighting** sebagai pembanding. Bedanya, cara ini nggak nambah data, tapi modelnya dikasih denda lebih besar kalau salah nangkep fraud. Di Random Forest lewat parameter class_weight="balanced", dan di XGBoost lewat scale_pos_weight yang nilainya rasio kelas mayoritas terhadap minoritas, yaitu 577,29.

Jadi total ada enam model yang dievaluasi: dua algoritma dikali tiga skenario, yaitu tanpa penanganan, SMOTE, dan class weighting. Skenario tanpa penanganan ini sengaja dipertahanin sebagai pembanding dasar biar kelihatan seberapa besar pengaruh penanganan imbalance-nya.

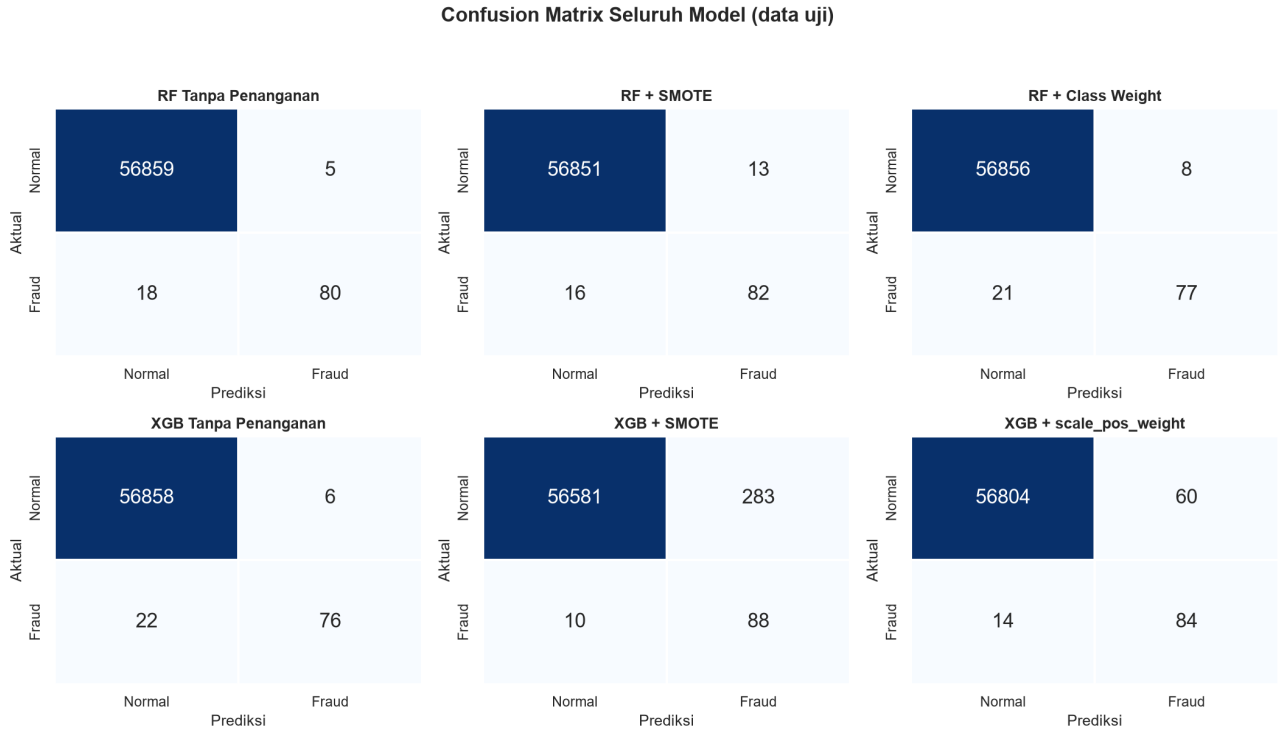
4. Hasil Evaluasi Model

Metrik yang dipakai: **precision** (dari yang diprediksi fraud, berapa yang beneran fraud), **recall** (dari yang beneran fraud, berapa yang ke-deteksi), **F1-score** (rata-rata harmonik keduanya), **ROC-AUC**, dan **PR-AUC**. Di data yang super timpang gini, PR-AUC justru lebih jujur daripada ROC-AUC karena dia fokus ke kelas minoritas, bukan ke performa di kelas mayoritas yang emang gampang.

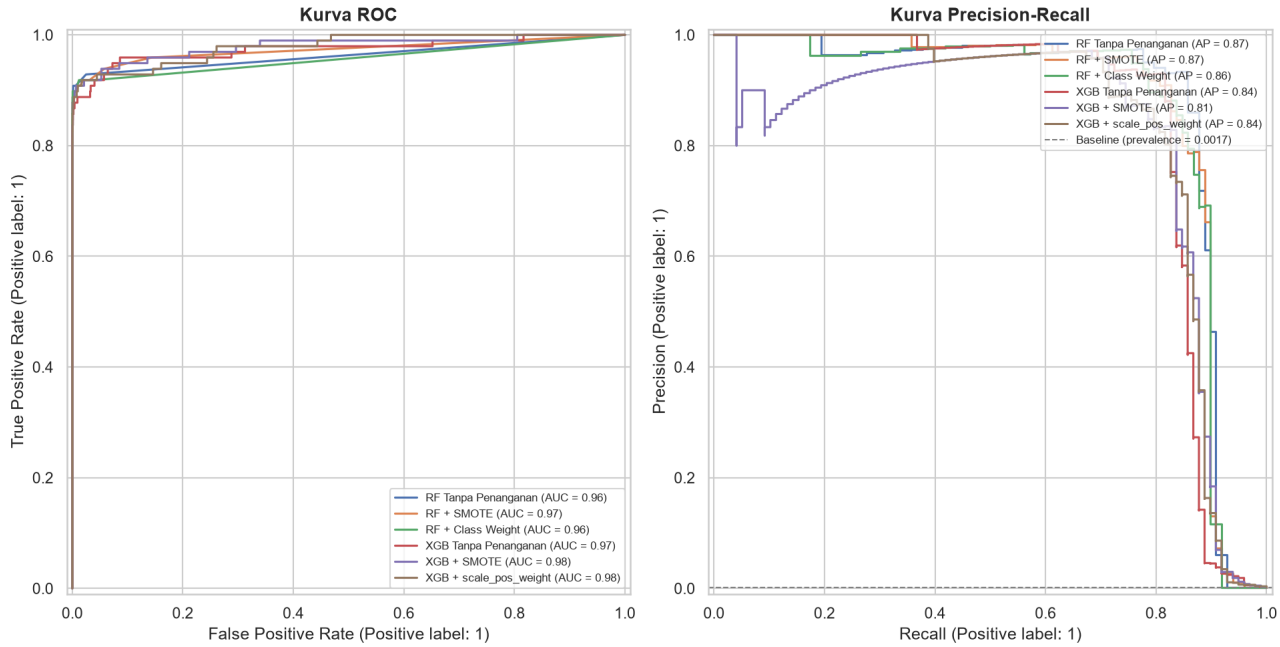
Model	Precision	Recall	F1	ROC-AUC	PR-AUC	Fraud kelewatan
RF Tanpa Penanganan	0,9412	0,8163	0,8743	0,9630	0,8734	18
RF + SMOTE	0,8632	0,8367	0,8497	0,9734	0,8706	16
RF + Class Weight	0,9059	0,7857	0,8415	0,9573	0,8629	21
XGB Tanpa Penanganan	0,9268	0,7755	0,8444	0,9748	0,8413	22
XGB + scale_pos_weight	0,5833	0,8571	0,6942	0,9789	0,8404	14
XGB + SMOTE	0,2372	0,8980	0,3753	0,9794	0,8115	10

Tabel 2. Perbandingan metrik keenam model pada data uji (56.864 normal dan 98 fraud).

Angka akurasinya nggak saya masukin ke tabel karena semuanya berkisar 99,49% sampai 99,96%, jadi hampir nggak ngebantu buat membedain model. Malah XGB + SMOTE yang akurasinya paling rendah (99,49%) justru punya recall tertinggi. Ini nunjukkin lagi kalau accuracy beneran nggak bisa dipakai sebagai patokan di dataset kayak gini.



Gambar 3. Confusion matrix keenam model. Kiri atas = TN, kanan atas = FP, kiri bawah = FN, kanan bawah = TP. Sel kiri bawah (FN) adalah fraud yang kelewatan.



Gambar 4. Kurva ROC (kiri) dan Precision-Recall (kanan). Di kurva ROC semua model kelihatan sama bagusnya, sedangkan di kurva PR perbedaannya lebih jelas.

Di kurva ROC kelihatan enam garisnya saling menumpuk di pojok kiri atas, jadi kelihatan seolah semua model sama bagusnya. Tapi di kurva Precision-Recall perbedaannya langsung kelihatan: garis XGB + SMOTE turun duluan, artinya dia harus nebak banyak transaksi sebagai fraud sebelum nangkep yang beneran.

5. Pembahasan

Penanganan imbalance nggak otomatis bikin modelnya lebih bagus. Ini temuan yang paling saya perhatikan di praktikum ini.

F1-score tertinggi justru ada di **Random Forest tanpa penanganan apa pun** (0,8743). Dia juga paling presisi (0,9412), jadi alarm palsunya paling sedikit, cuma 5 dari 56.864 transaksi normal. Tapi recall-nya cuma 0,8163, artinya **18 kasus fraud kelewatan**.

Begitu dikasih **SMOTE**, recall naik dari 0,8163 ke 0,8367 dan ROC-AUC naik dari 0,9630 ke 0,9734. Fraud yang kelewatan turun dari 18 jadi 16. Harganya precision turun ke 0,8632 dan alarm palsu naik dari 5 jadi 13. Jadi SMOTE di sini beneran ngefek, cuma dampaknya nggak sebesar yang mungkin kebayang awalnya.

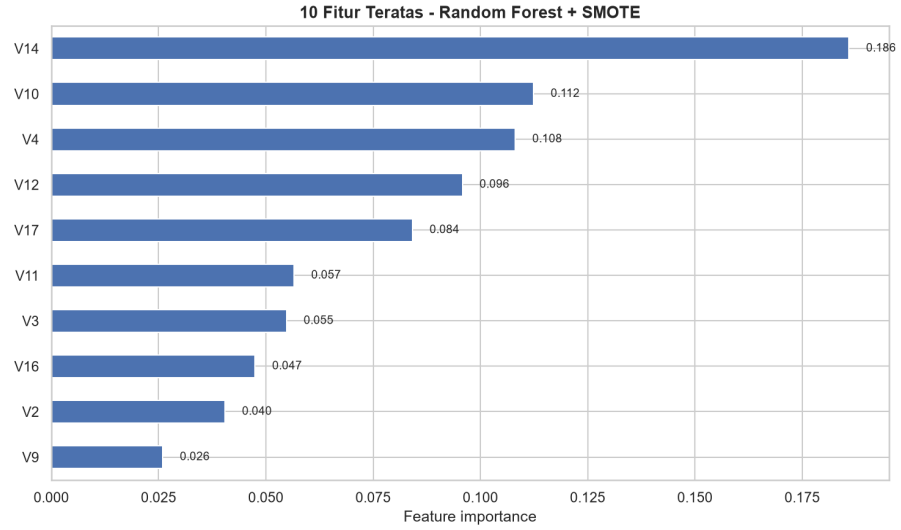
Class weighting justru paling lemah di kelompok Random Forest. Recall-nya malah turun ke 0,7857, lebih buruk dari RF yang tanpa penanganan sama sekali. Ini yang bikin saya mikir, cara paling murah (tinggal set parameter) belum tentu yang paling cocok buat setiap dataset.

XGB + SMOTE paling agresif. Recall-nya 0,8980, cuma 10 fraud yang lolos, terbaik di antara semua model. Tapi precision-nya anjlok ke 0,2372. Artinya dari 371 transaksi yang dia tandai fraud, 283 di antaranya ternyata normal. F1-nya jatuh ke 0,3753 dan MCC-nya juga paling rendah (0,4601). Angka recall-nya emang bagus, tapi di balik itu ada ratusan alarm palsu yang harus ditelusuri tim investigasi.

Alasannya cukup masuk akal. SMOTE bikin 227.077 sampel fraud sintetis dari cuma 394 fraud asli. Buat Random Forest yang ngandelin voting banyak pohon, tambahan sampel segitu bikin keputusannya lebih stabil. Tapi buat XGBoost yang tiap pohonnya belajar berurutan dari kesalahan pohon sebelumnya, distribusi yang dibikin imbang itu malah bikin dia kebanyakan ngasih skor tinggi ke transaksi normal yang mirip fraud, jadi threshold-nya jadi terlalu longgar.

Pilihan model. Kalau lihat dari sisi kerugian, fraud yang lolos itu cost-nya jauh lebih besar daripada false alarm, yang satu langsung jadi uang hilang, yang satu cuma bikin tim fraud repot ngecek. Berdasar itu, **Random Forest + SMOTE** adalah yang paling seimbang buat dipakai: recall 0,8367, ROC-AUC 0,9734, dan PR-AUC 0,8706 yang cuma selisih 0,003 dari RF tanpa penanganan. Dia nangkep 2 fraud lebih banyak dari RF biasa, dan ranking probabilitasnya juga lebih bagus.

Tapi kalau kebijakannya “nggak boleh ada fraud yang lolos sama sekali”, **XGB + SMOTE** bisa jadi pilihan karena cuma ngelewat 10 kasus. Konsekuensinya tim investigasi harus siap nanganin ratusan alarm palsu tiap periode. Jadi pilihannya bukan model terbaik secara absolut, tapi mana yang konsekuensinya paling bisa ditanggung.



Gambar 5. Sepuluh fitur dengan importance tertinggi pada model Random Forest + SMOTE.

Terakhir, dari sisi fitur kelihatan bahwa **V14, V10, V4, V12, dan V17** yang paling banyak ngefek ke keputusan model. Lima fitur itu doang udah nyumbang lebih dari 58 persen total importance. Kolom **Amount** sendiri justru nggak masuk sepuluh besar, yang berarti nilai transaksinya bukan penanda utama fraud di dataset ini.

6. Kesimpulan

1. Distribusi kelas di dataset ini timpang parah (0,17%), dan itu bikin accuracy jadi metrik yang nggak bermakna. Model dummy yang nggak belajar apa-apa aja dapet akurasi 99,83% tapi recall-nya 0.

2. SMOTE berhasil ngingetin recall, tapi dampaknya beda-beda tiap model. Di Random Forest hasilnya masih masuk akal, di XGBoost justru bikin precision hancur ke 0,2372.

3. Dari enam model yang dicoba, Random Forest + SMOTE paling seimbang antara recall (0,8367), precision (0,8632), dan PR-AUC (0,8706).

4. Resampling yang cuma dilakukan di data latih bikin hasil evaluasinya lebih bisa dipercaya, karena data uji tetep pakai distribusi asli.

5. Class weighting nggak selalu lebih baik dari SMOTE. Di percobaan ini dia justru paling lemah di kelompok Random Forest, dengan recall 0,7857.

6. Fitur V14, V10, V4, V12, dan V17 adalah yang paling ngefek, sedangkan Amount justru nggak masuk sepuluh besar.